

Partial Differential Equation Agents (PDE-Agents): A Large Language Model-Orchestrated Multi-Agent Framework for Automated Finite Element Simulations with Knowledge Graph-Augmented Reasoning^{*}

Sayan Adhikari^{a,*}, Gulshan Noorsumar^a, Øyvind Jensen^a

^a*Institute for Energy Technology (IFE), Kjeller, Norway*

Abstract

We present *PDE-Agents*, a multi-agent ecosystem that automates the full lifecycle of partial differential equation (PDE) / finite element method (FEM) simulations through natural-language interaction. The system combines three specialist large language model (LLM) agents—Simulation, Analytics, and Database—orchestrated via a LangGraph supervisor graph, with a locally-deployed open-source LLM stack (Qwen3-Coder-Next, Llama 4 Scout) running on dual NVIDIA RTX PRO 6000 Blackwell graphics processing units (GPUs) with approximately 196 gigabytes of combined video memory. The architecture is model-agnostic; we report cross-model validation across two generations of open-source LLMs. A central novelty is the integration of a *graph-based retrieval-augmented generation* (GraphRAG) knowledge base (Neo4j + 768-dimensional vector embeddings) that enriches agents with curated material properties, known failure patterns, and prior run lineage. We

^{*}Code: <https://github.com/MatPro-IFE/pde-agents>

^{*}Corresponding author

Email address: `sayan.adhikari@ife.no` (Sayan Adhikari)

report seven empirical contributions: (i) a formal verification and validation (V&V) study confirming second-order spatial convergence ($\mathcal{O}(h^2)$) for two non-trivial benchmark cases and algebraic exactness for the linear case on the heat equation solver; (ii) a three-way ablation study across 50 benchmark tasks with a frozen knowledge graph (KG), comparing KG On, KG Off, and KG Smart, showing that KG Smart achieves 100% success *and* the highest output quality (physics score 0.933 vs. 0.853 for KG Off, material property fidelity (MPF) 0.926 vs. 0.796), and that our *KG Smart* integration, combining warm-start injection with lazy conditional retrieval, matches KG Off reliability while maximising output fidelity; (iii) a novel-material experiment using three fictional materials whose properties exist only in the KG, where KG Smart achieves near-perfect MPF = 1.00 versus 0.34 for the KG-free baseline; (iv) a failure analysis tracing KG On’s 3 systematic failures to budget exhaustion and timeout, establishing warm-start injection as the dominant factor in KG Smart’s reliability advantage; (v) an adaptive KG decision framework (Algorithm 1) that selects the optimal retrieval mode per task; (vi) production-scale agent quality metrics from 1,369 real simulation runs showing 97.8% overall success and an 85.4% first-try rate; and (vii) a controlled 100-task KG growth experiment showing that accumulated run history yields a difficulty-dependent quality gain, with hard-task MPF improving by 8.8% between passes while easy and novel tasks remain at ceiling. All code, models, and evaluation artifacts are released openly. Taken together, these results point to *integration pattern*, rather than knowledge content, as what decides whether GraphRAG augmentation helps or hinders an LLM agent, and they yield concrete design principles for autonomous FEM simulation assistants.

Keywords: large language model agents, finite element method, knowledge graph, graph-based retrieval-augmented generation, multi-agent systems, scientific computing automation

1. Introduction

Finite element method simulations are central to engineering analysis across structural mechanics, heat transfer, fluid dynamics, and electromagnetics. Yet the gap between a domain expert’s intent—“what if I use aluminium instead of steel?”—and a running, verified simulation involves a sequence of error-prone manual steps: geometry creation, boundary condition specification, solver parameter selection, result interpretation, and iterative debugging.

The recent emergence of *reasoning-capable* LLMs [1, 2, 3] has opened the possibility of agents that can autonomously traverse this workflow. Prior work on “AI for scientific computing” has largely focused on surrogate modelling [4, 5, 6], operator learning [7], or dataset-specific fine-tuning [8]. Much less attention has been paid to *procedural automation*—using LLMs to *set up and manage* solvers rather than to replace them.

Meanwhile, the *retrieval-augmented generation* (RAG) paradigm [9] has demonstrated that grounding LLM outputs with authoritative external knowledge dramatically reduces hallucination. The graph-structured variant, GraphRAG [10], further enables multi-hop reasoning over relational knowledge—particularly relevant in engineering contexts where material properties, mesh requirements, and solver stability rules form a rich, interconnected knowledge base.

In this paper we make the following contributions:

- 22 1. We design and implement *PDE-Agents*, a complete, containerised multi-
23 agent ecosystem for automated FEM simulation (section 3).
- 24 2. We describe a novel *GraphRAG integration pattern* that encodes material
25 properties, known failure modes, and prior run lineage as a Neo4j
26 knowledge graph with Hierarchical Navigable Small World (HNSW)
27 vector-indexed embeddings [11] (section 4).
- 28 3. We conduct a rigorous V&V study that confirms $\mathcal{O}(h^2)$ spatial conver-
29 gence for two non-trivial benchmark cases (transient Fourier and steady
30 Poisson) and algebraic exactness for the linear profile case, validating
31 the numerical kernel against analytical solutions (section 5).
- 32 4. We run a three-way ablation study (KG On, KG Off, KG Smart) over
33 50 tasks with a frozen knowledge graph. KG-enabled modes produce
34 higher-quality simulation outputs (physics score, material property
35 fidelity) than the KG-free baseline, and *KG Smart* reaches 100% success
36 and the highest output quality (section 6).
- 37 5. We introduce a *novel-material experiment* using three fictional materials
38 absent from any LLM training corpus. KG Smart attains $\approx 2.9\times$ the
39 material property fidelity of the KG-free baseline, which fabricates
40 physically incorrect properties (section 6.4).
- 41 6. We trace KG On’s 3 systematic failures to budget exhaustion and
42 timeout, revealing that warm-start injection, not lazy retrieval, is the
43 dominant factor in KG Smart’s reliability advantage (section 6.5).
- 44 7. We report production-scale agent quality metrics derived from 1,369 real
45 simulation runs, together with a controlled 100-task KG growth experi-
46 ment in which the quality gain proves difficulty-dependent: hard-task

47 MPF improves by 8.8% as the KG accumulates run history (section 7).

48 2. Related Work

49 *LLM agents for scientific computing..* ChemCrow [12] demonstrated that
50 equipping an LLM with chemistry tools (RDKit, reaction planners) enables
51 autonomous laboratory-scale reasoning. SciAgent [13] and similar systems
52 extend this to broader scientific question answering. Our work targets *numer-*
53 *ical simulation* rather than knowledge retrieval, requiring tool-call pipelines
54 that produce and verify deterministic numerical outputs.

55 *Autonomous FEM systems..* FEniCS [14] and its successor DOLFINx/FEn-
56 iCSx [15] provide Python-scriptable FEM solvers that are amenable to LLM-
57 driven automation. Early work on “simulation copilots” [16, 17] shows that
58 LLMs can generate solver scripts from natural language with moderate success,
59 but these approaches are single-turn and lack closed-loop debugging. More
60 recently, ALL-FEM [18] fine-tunes LLMs on 1,000+ verified FEniCS scripts
61 and embeds them in a multi-agent workflow achieving 71.8% code-level suc-
62 cess on multiphysics benchmarks, and FEABench [19] provides a systematic
63 evaluation using COMSOL Multiphysics (88% executable API calls). PDE-
64 Agents differs by using unmodified open-source LLMs with knowledge-graph
65 augmentation rather than domain-specific fine-tuning, and by providing a
66 controlled ablation of retrieval strategies with physics-aware quality metrics.

67 *Multi-agent orchestration..* LangGraph [20] implements graph-structured
68 agent workflows that support conditional routing and persistent state, enabling
69 the supervisor pattern we adopt. AutoGen [21] and CrewAI [22] offer related

70 frameworks; we choose LangGraph for its explicit state graph semantics and
71 tight LangChain integration.

72 *Retrieval-augmented generation for engineering..* RAG [9] has been applied
73 to engineering documentation retrieval [23], code generation [24], and more
74 recently to simulation parameter suggestion [25]. GraphRAG [10] adds struc-
75 tured relational traversal. Corrective RAG (CRAG) [26] introduces adaptive
76 retrieval decisions based on document relevance scoring, and AriGraph [27]
77 demonstrates episodic knowledge-graph memory for LLM agents. Our *KG*
78 *Smart* pattern draws on both: warm-start injection from similar past runs
79 (episodic memory) with lazy conditional tool invocation (corrective retrieval).
80 To our knowledge, ours is the first system to combine GraphRAG with a live,
81 iterative FEM simulation loop.

82 *Knowledge graphs for materials science..* Materials-focused knowledge graphs
83 (e.g. MatKG [28], OPTIMADE [29]) encode material properties in RDF/OWL
84 formats. Buehler [30] demonstrates that retrieval-augmented ontological KG
85 strategies outperform flat RAG for materials design by capturing mechanistic
86 relationships between concepts. Our Neo4j KG is lighter-weight and purpose-
87 built for run-time agent queries, prioritising latency over completeness.

88 3. System Architecture

89 3.1. Design Philosophy

90 PDE-Agents adopts four guiding principles:

- 91 1. **Open and local:** All models run locally via Ollama on dual NVIDIA

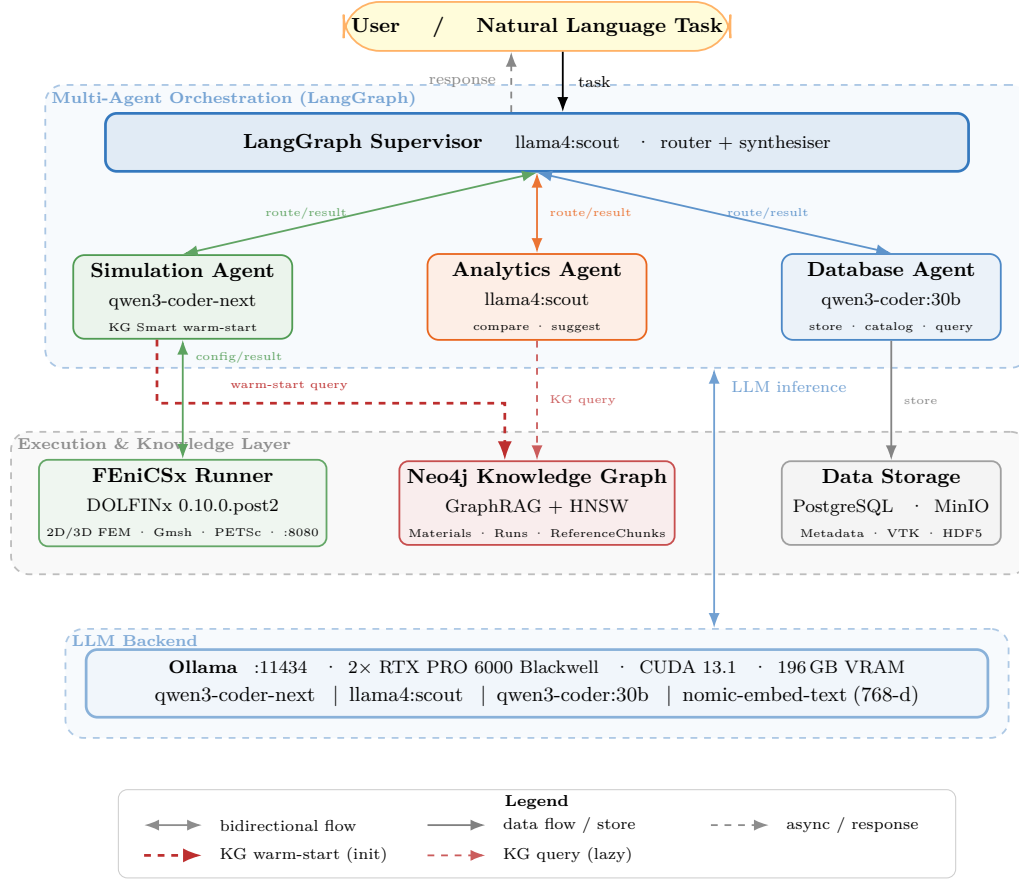


Figure 1: System architecture of PDE-Agents (four-tier layout). *Tier 1*: user natural-language interface. *Tier 2*: Multi-Agent Orchestration — a LangGraph Supervisor routes tasks to three specialist agents (Simulation, Analytics, Database), each backed by a locally-deployed LLM via Ollama. *Tier 3*: Execution & Knowledge — FEniCSx FEM runner (DOLFINx 0.10.0.post2), Neo4j GraphRAG Knowledge Graph (HNSW-indexed 768-d embeddings), and persistent storage (PostgreSQL + MinIO). *Tier 4*: shared Ollama LLM Backend (196 GB VRAM, CUDA 13.1). The thick dashed red arrow (Simulation Agent $\rightarrow$ Neo4j) represents the *KG Smart warm-start*: the top-3 similar past runs are retrieved via HNSW vector search and injected into the agent’s prompt before the reasoning loop. The thin dashed red arrow (Analytics Agent $\rightarrow$ Neo4j) is the standard lazy KG query path. The blue bidirectional arrow at right carries LLM inference traffic between the orchestration layer and the shared Ollama backend.

- 92 RTX PRO 6000 Blackwell GPUs (≈ 196 GB VRAM, CUDA 13.1), elim-
 93 inating API costs and data-privacy concerns.
- 94 2. **Verifiable:** Every agent reasoning step, tool call, and result is logged
 95 to a relational database, enabling post-hoc audit.
- 96 3. **Containerised:** All 10 services are orchestrated via Docker Compose,
 97 ensuring reproducible deployment.
- 98 4. **Separation of concerns:** The FEM solver, LLM stack, knowledge
 99 graph, and persistence layer are decoupled services communicating over
 100 a private bridge network.

101 3.2. Agent Stack

102 Each agent’s LLM backend is configurable via environment variables,
 103 enabling drop-in model upgrades without code changes. table 1 lists the
 104 default models and the alternatives validated in the cross-model experiments
 105 of section 6.4.

Table 1: Agent model configuration. Current defaults reflect the model upgrade from the initial (v1) deployment. The ablation study uses a frozen KG snapshot across 50 benchmark tasks per mode.

Agent	Env Variable	Default (current)	Previous (v1)
Orch.	ORCHESTRATOR_MODEL	llama4:scout	llama3.3:70b
Sim.	SIM._AGENT_MODEL	qwen3-coder-next	qwen2.5-coder:32b
Anal.	ANAL._AGENT_MODEL	llama4:scout	llama3.3:70b
DB	DB_AGENT_MODEL	qwen3-coder:30b	qwen2.5-coder:14b

106 *Orchestrator..* A LangGraph **StateGraph** supervisor receives the user’s natural-
 107 language request and routes it to one of three specialist agents via structured

108 JSON decisions. After each agent reports its result the supervisor either
109 routes to another agent or synthesises a final response.

110 *Simulation Agent..* The core reasoning agent, executing a ReAct loop [3]
111 with up to 25 reasoning steps and nine tools: `check_config_warnings`,
112 `query_knowledge_graph`, `validate_config`, `run_simulation`, `modify_config`,
113 `debug_simulation`, `list_recent_runs`, `get_run_status`, `run_parametric_sweep`.

114 *Analytics Agent..* Max 12 iterations; tools for statistical comparison across
115 runs, sensitivity analysis, and LLM-generated textual summaries.

116 *Database Agent..* Max 10 iterations; answers history queries against Post-
117 greSQL, cataloguing results and tracing run lineage.

118 3.3. Tool-Call Compatibility

119 Supporting multiple LLM families requires robust tool-call parsing, as
120 models encode tool invocations differently. The Qwen 2.5-Coder family emits
121 tool calls as JSON text in the `content` field rather than in the structured
122 `tool_calls` field expected by LangChain’s `ToolNode`. We implement a four-
123 pass normalisation procedure in `BaseAgent._parse_content_tool_call`:
124 (1) attempt `json.loads` on the full content; (2) extract from markdown
125 fences; (3) extract the first `{...}` block; (4) repair truncated JSON via
126 progressive bracket-closing and regex fallback.

127 A separate compatibility issue emerged with Qwen3-Coder-Next: the
128 model occasionally *double-encodes* JSON tool arguments (e.g. `"{"key\":"`
129 `\val\"}"`). All tool functions now use a `_safe_json_parse` helper that
130 detects and recursively unwraps such double-encoded strings, so the same

131 tool code works across model families without model-specific workarounds.
132 Llama 4 Scout, by contrast, uses the standard `tool_calls` field natively and
133 requires no special parsing.

134 3.4. FEM Solver: DOLFINx Heat Equation

135 The FEniCSx runner container (DOLFINx 0.10.0.post2) exposes a FastAPI
136 endpoint that accepts a `HeatConfig` JSON and returns a result including run
137 metadata. The solver uses P1 (linear Lagrange) elements, with Backward
138 Euler ($\theta = 1$, unconditionally stable) as the default time integration scheme,
139 supporting Dirichlet, Neumann, and Robin boundary conditions. Geometry
140 is built with Gmsh [31] for 2D and 3D domains from a library of nine param-
141 eterised types: `rectangle`, `l_shape`, `circle`, `annulus`, `hollow_rectangle`,
142 `t_shape`, `stepped_notch`, `box`, and `cylinder`; Gmsh geometries use named
143 physical groups for boundary identification, enabling arbitrary complex do-
144 main topologies.

145 4. Knowledge Graph Architecture

146 The Neo4j knowledge graph [32] (Neo4j 5 Community Edition) serves
147 three functions:

- 148 1. **Material property lookup.** A curated library of materials (met-
149 als, ceramics, polymers) with thermal conductivity k , density ρ , and
150 specific heat c_p ranges, sourced from engineering handbooks and val-
151 idated against ASM International data. Approximate values: steel
152 $k \approx 50 \text{ W/mK}$, copper $k \approx 385 \text{ W/mK}$, aluminium $k \approx 205 \text{ W/mK}$.

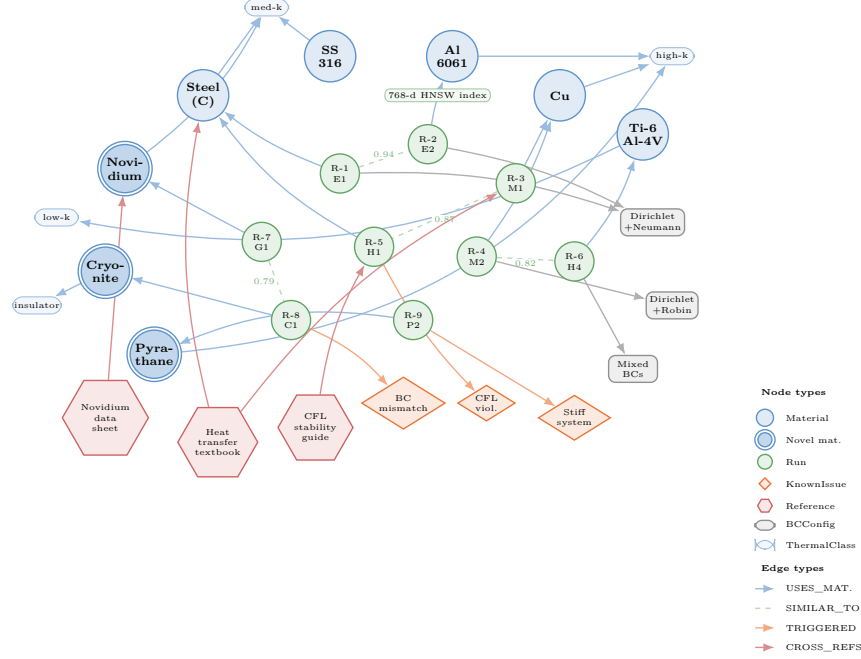


Figure 2: Knowledge graph visualisation (Neo4j-style). **Blue circles:** material nodes (double border = novel/fictional materials known only to the KG). **Green circles:** simulation run nodes linked to their materials via `USES_MATERIAL` edges and to each other via dashed `SIMILAR_TO` edges (cosine similarity on 768-d HNSW embeddings; scores shown on edges). **Orange diamonds:** known-issue nodes triggered by failed runs. **Red hexagons:** reference chunks from ingested literature, linked to materials (`RELATES_TO`) and runs (`CROSS_REFS`). **Grey boxes:** boundary-condition pattern nodes. Pill-shaped nodes show the thermal-class taxonomy.

153 2. **Failure pattern catalogue.** `KnownIssue` nodes encode failure modes
 154 observed during system operation. A pure-Python rule engine (`rules.py`)
 155 fires nine pre-run checks analytically before each simulation:

Rule code	Trigger
INCONSISTENT_IC	$ u_{\text{init}} - T_{\text{BC,min}} > 100 \text{ K}$
EXPLICIT_CFL_VIOLATION	$\theta < 0.5$ and $\Delta t > h^2/(2\alpha)$
NEAR_EXPLICIT_SCHEME	$0 \leq \theta < 0.5$
COARSE_MESH_2D	$n_x < 10$ or $n_y < 10$
COARSE_MESH_3D	any direction < 8
SHORT_SIMULATION	$t_{\text{end}} < 5 \Delta t$
INVALID_MATERIAL_PROPS	k, ρ or $c_p \leq 0$
LARGE_DT_RELATIVE_DIFFUSION	$\Delta t > 10 h^2/\alpha$
NO_BOUNDARY_CONDITIONS	<code>bcs</code> list empty

157 3. **Run lineage and semantic retrieval.** Each completed `Run` node is
 158 embedded with `nomic-embed-text` [33] (768 dimensions) and indexed
 159 in an HNSW vector index [11]. Up to $k=5$ `SIMILAR_TO` edges are created
 160 to the nearest neighbours with cosine similarity ≥ 0.85 , precomputing
 161 the neighbourhood graph for fast agent traversal. The Simulation Agent
 162 can query “find similar past runs” to retrieve configurations with known
 163 outcomes, enabling few-shot in-context learning from the system’s own
 164 history.

165 *Document intelligence pipeline..* A Celery-backed ingestion pipeline processes
 166 PDF papers, standards documents, and HTML references using Docling [34]
 167 for structured extraction, splitting them into `ReferenceChunk` nodes with

context-aware 256-token windows (overlap 32 tokens) and HNSW-indexed embeddings. Each chunk is automatically cross-referenced to semantically similar Run nodes via CROSS_REFS edges (cosine ≥ 0.78), bridging literature knowledge to simulation history. As a representative scale point: indexing the 50-page FEniCSx tutorial produced 298 chunks and 1,073 cross-references.

5. Verification and Validation

We conduct a formal V&V study following ASME V&V 10-2006 guidelines [35], comparing the DOLFINx FEM solver against closed-form analytical solutions on three benchmark cases.

5.1. Benchmark Cases

Case 1: 2D Steady-State Linear Profile.. Laplace equation on $\Omega = [0, 1]^2$ with Dirichlet conditions $T|_{x=0} = 0$, $T|_{x=1} = 1$ and Neumann zero-flux on top/bottom. Analytical solution: $T(x, y) = x$. P1 elements are nodally exact for linear solutions; we use UFL spatial-coordinate expressions in high-order quadrature (degree 8) to measure the true continuous L^2 error.

Case 2: 2D Transient Fourier Mode Decay.. Heat equation $\partial_t T = \alpha \nabla^2 T$ with homogeneous Dirichlet on all boundaries and initial condition $T_0 = \sin(\pi x) \sin(\pi y)$. Analytical solution: $T(x, y, t) = e^{-2\alpha\pi^2 t} \sin(\pi x) \sin(\pi y)$.

Case 3: 2D Steady-State Poisson with Constant Source.. $-k \nabla^2 T = f$ with $f = 1000 \text{ W/m}^3$, Dirichlet $T = 0$ on left/right, Neumann on top/bottom. Exact solution: $T(x) = f(x - x^2)/(2k)$.

189 *5.2. Convergence Study Protocol*

190 For each case we solve at five mesh resolutions $N \in \{8, 16, 32, 64, 128\}$
 191 (DOFs $\approx 81, 289, 1089, 4225, 16641$) and compute:

$$\|e\|_{L^2} = \left(\int_{\Omega} (u_h - u_{\text{exact}})^2 dx \right)^{1/2} \quad (1)$$

192 using degree-8 quadrature applied to UFL `SpatialCoordinate` expressions
 193 (not to interpolated nodal values, which would give spurious nodal exactness
 194 for Case 1). The convergence rate is estimated by linear regression on the
 195 log-log plot.

196 *5.3. Results*

Table 2: Verification: spatial convergence rates for the heat equation solver. Expected rate is $\mathcal{O}(h^2)$ for P_1 elements.

Case	Description	DOFs	$\ e\ _{L^2}$	Rate
Steady Linear	Linear $T(x, y) = x$; exact in P_1 space	16 641	2.47×10^{-9}	<i>exact</i>
Transient Fourier	$e^{-2\pi^2 t} \sin(\pi x) \sin(\pi y)$ de- cay mode	16 641	1.50×10^{-5}	2.04
Steady Poisson	$-\nabla^2 u = 1$; closed-form polynomial soln.	16 641	5.57×10^{-6}	2.00

197 All three cases pass: Cases 2 and 3 achieve rates of 2.04 and 2.00, consistent
 198 with the theoretical $\mathcal{O}(h^2)$ prediction for P_1 finite elements on uniform
 199 Cartesian meshes. Case 1 achieves machine-precision errors (order 10^{-9})
 200 because the linear profile lies exactly in the P_1 polynomial space, confirming
 201 correct solver implementation.

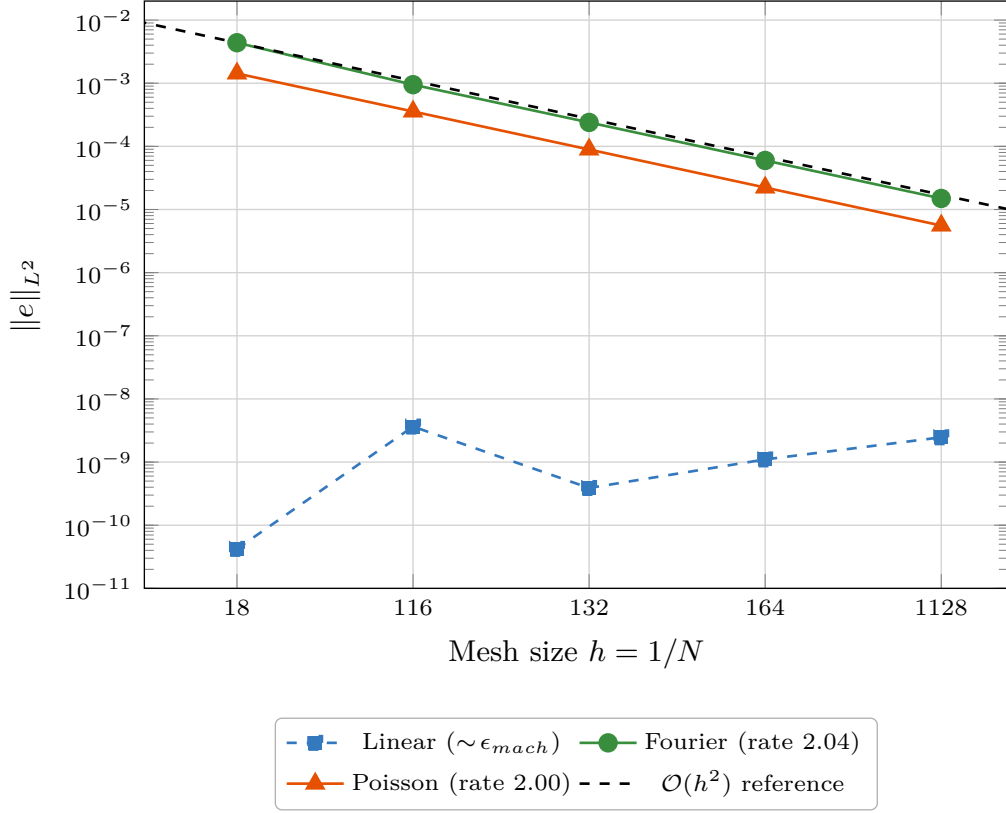


Figure 3: Spatial convergence study. Cases 2 and 3 exhibit the expected $\mathcal{O}(h^2)$ rate for P1 elements. Case 1 is algebraically exact (linear profile is represented exactly in the P1 space), with $\|e\|_{L^2} \approx \mathcal{O}(10^{-9})$ attributable only to floating-point rounding.

202 5.4. Representative Simulation Gallery

203 Beyond numerical convergence, a practical simulation assistant needs
204 breadth: it must handle diverse materials, boundary condition types, do-
205 main geometries, time dependence, and dimensionality. Figure 4 presents six
206 representative heat-transfer problems, each solved end-to-end by the Simula-
207 tion Agent from a single natural-language prompt. No manual intervention
208 was required for mesh generation, solver configuration, boundary condition
209 application, or post-processing. All cases use P1 Lagrange elements on tri-
210 angular (2D) or tetrahedral (3D) meshes, solved by DOLFINx 0.10.0.post2.
211 Temperature fields are reported in Kelvin throughout for consistency.

212 *Materials..* The gallery covers five engineering metals whose thermal con-
213 ductivities span two orders of magnitude: copper ($k = 385 \text{ W}/(\text{m}\cdot\text{K})$, panel a),
214 AISI 1010 steel ($k = 50 \text{ W}/(\text{m}\cdot\text{K})$, panel b), aluminium 6061 ($k = 205 \text{ W}/(\text{m}\cdot\text{K})$,
215 panels d and e), Ti-6Al-4V titanium ($k = 6.7 \text{ W}/(\text{m}\cdot\text{K})$, panel c), and
216 stainless steel 304 ($k = 16.3 \text{ W}/(\text{m}\cdot\text{K})$, panel f). In a production deploy-
217 ment, the agent retrieves these values from the Neo4j knowledge graph via
218 `query_knowledge_graph`.

219 *Boundary conditions..* Panel (a) applies the simplest configuration: Dirich-
220 let conditions on opposing faces ($T = 373.15 \text{ K}$ left, $T = 273.15 \text{ K}$ right),
221 producing the expected linear gradient. Panel (b) combines all four major
222 BC types on a single domain: Dirichlet (300 K, left), Neumann inward flux
223 ($2 \text{ kW}/\text{m}^2$, top), Robin convection ($h = 25 \text{ W}/(\text{m}^2\cdot\text{K})$, $T_\infty = 293 \text{ K}$, right),
224 and insulated (bottom), together with volumetric heating $Q = 5 \text{ kW}/\text{m}^3$,
225 creating the asymmetric field visible in the plot. Panel (c) uses Dirichlet BCs

on opposing edges (573 K left, 293 K right) with the insulated hole boundary
distorting the field into characteristic thermal concentration bands. Panel (f)
combines Dirichlet (300 K bottom, 600 K left face) with Robin convection on
the top face of a 3D cube.

Geometry.. Panels (a), (b), and (d) use structured Cartesian meshes on
the unit square. Panels (c) and (e) use Gmsh-based meshing of non-trivial
geometries: a plate with a circular cutout (Gmsh boolean difference, $r = 0.15$,
 $\approx 5\,000$ vertices) and an L-shaped domain ($[0, 1]^2 \setminus [0.5, 1]^2$, mesh size ≈ 0.02).
Panel (f) meshes a unit cube with 24^3 tetrahedra ($\approx 15\,600$ vertices), exercising
the 3D path.

Time dependence.. Panel (e) solves the transient heat equation using 100
implicit-Euler steps ($\Delta t = 0.005$ s) on the L-shaped domain (aluminium,
 $\rho = 2700$ kg/m³, $c_p = 900$ J/(kg·K)), with the snapshot at $t = 0.50$ s showing
the thermal front propagating from the hot left edge.

Source terms.. Panel (d) features a localised Gaussian volumetric heat source
 $Q(x, y) = Q_{\text{peak}} \exp\left(-\frac{(x-x_0)^2 + (y-y_0)^2}{2\sigma^2}\right)$ with $Q_{\text{peak}} = 5 \times 10^6$ W/m³, centred
at (0.3, 0.7) with $\sigma = 0.08$, and Dirichlet boundaries at 293 K. This setup
is representative of laser spot heating and Joule-heating applications, and
produces the characteristic concentric isotherms visible in the figure.

Reproducibility.. Each case is implemented as a self-contained Python script
in `evaluation/examples/` (see Table 3). All simulation parameters (material
properties, mesh resolution, boundary values, time stepping) are defined at
the top of each script, making it straightforward for reviewers to modify
and re-run individual cases. Running `make eval-examples` executes all six

cases inside the `pde-fenics` Docker container and regenerates the composite figure. Intermediate results are saved as NumPy `.npz` archives containing mesh coordinates, cell connectivity, and solution vectors, enabling figure regeneration without re-running the solver.

Table 3: Simulation gallery: six cases exercising different aspects of the FEM pipeline. Scripts in `evaluation/examples/`.

	Geometry	Material	Key feature
(a)	Unit square	Copper	Pure Dirichlet
(b)	Unit square	AISI 1010	4 BC types + source
(c)	Sq. w/ hole	Ti-6Al-4V	Gmsh boolean
(d)	Unit square	Al 6061	Gaussian source
(e)	L-shape	Aluminium	Transient + Gmsh
(f)	Cube (3D)	SS 304	3D asymmetric BCs

6. Ablation Study: Knowledge Graph Contribution

6.1. Experimental Design

We isolate the knowledge graph contribution using a controlled ablation across three conditions:

- **KG On** (mandatory): Full system with `check_config_warnings` and `query_knowledge_graph` tools enabled; the system prompt requires their use before every run.

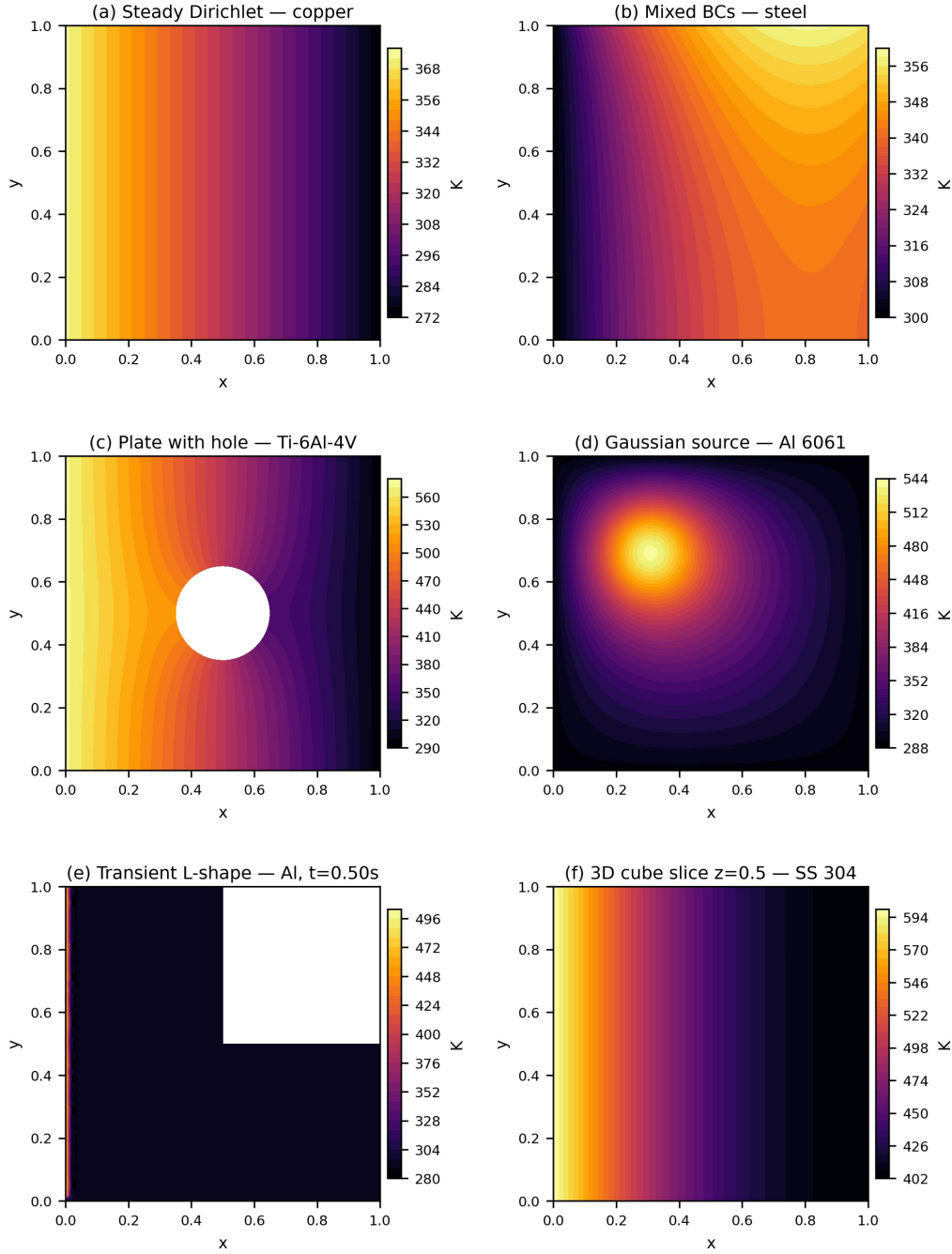


Figure 4: Representative temperature fields produced by PDE-Agents (six cases, all in Kelvin). **(a)** Steady-state Dirichlet on copper ($k = 385 \text{ W}/(\text{m}\cdot\text{K})$; $T = 373.15 \text{ K}$ left, $T = 273.15 \text{ K}$ right; $N = 64$). **(b)** Mixed BCs on AISI 1010 steel ($k = 50 \text{ W}/(\text{m}\cdot\text{K})$): Dirichlet 300 K left, Neumann $2 \text{ kW}/\text{m}^2$ top, Robin $h = 25 \text{ W}/(\text{m}^2\cdot\text{K})$ right, insulated bottom, $Q = 5 \text{ kW}/\text{m}^3$. **(c)** Plate with circular hole ($r = 0.15$) in Ti-6Al-4V ($k = 6.7 \text{ W}/(\text{m}\cdot\text{K})$); Gmsh boolean difference; Dirichlet $573 \text{ K}/293 \text{ K}$ on opposing edges. **(d)** Localised Gaussian heat source on Al 6061 ($Q_{\text{peak}} = 5 \times 10^6 \text{ W}/\text{m}^3$, $\sigma = 0.08$, centre $(0.3, 0.7)$); Dirichlet 293 K on all boundaries. **(e)** Transient L-shaped domain in aluminium at $t = 0.50 \text{ s}$ (implicit

- 261 • **KG Off** (baseline): Identical system with KG tools removed entirely;
262 the agent proceeds directly to validation and execution.
- 263 • **KG Smart** (proposed): KG tools remain available but the system
264 prompt instructs *lazy* use (only after failures or for unknown mate-
265 rials). Before the agent loop, the task description is embedded via
266 `nomic-embed-text` and the top-3 most similar past successful runs
267 are injected into the system prompt as few-shot reference examples
268 (warm-start).

269 fig. 5 contrasts the agent workflow under each condition. KG Off proceeds
270 directly from parsing to simulation; KG On forces two KG round-trips before
271 every attempt (and loops on failure); KG Smart front-loads a single HNSW
272 warm-start and defers KG queries to the failure-recovery path only.

273 *Methodology..* To eliminate data leakage between conditions, we *freeze* the
274 knowledge graph (set `KG_READ_ONLY=true` at the code level) before starting
275 the ablation: all three modes read from the same KG snapshot, but no mode
276 can write to it, ensuring that a failure in one mode cannot help subsequent
277 modes. Tasks are shuffled independently per mode and run in-process inside
278 the agents container to eliminate HTTP overhead. Confidence intervals use
279 Wilson scores [36]; pairwise comparisons use chi-squared tests and Cohen’s *h*
280 / Cohen’s *d* effect sizes [37].

281 *Benchmark tasks..* Fifty tasks span four difficulty levels:

- 282 • **Easy (12 tasks)**: Explicit numerical parameters, straightforward setup.

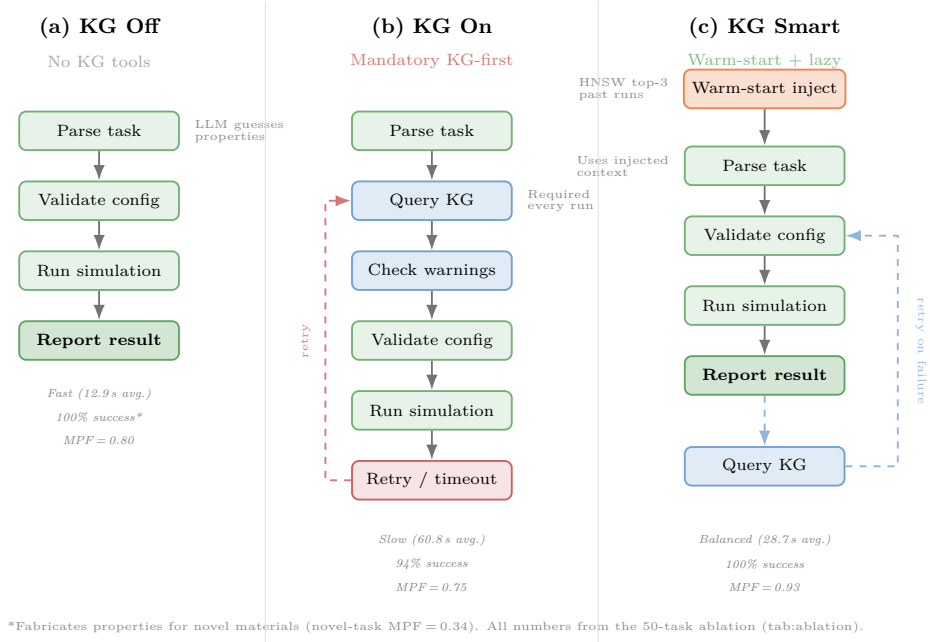


Figure 5: Agent workflow under the three KG integration modes. **(a)** KG Off: fast but the LLM fabricates material properties. **(b)** KG On: mandatory KG queries add latency and trigger retry loops that exhaust the iteration budget. **(c)** KG Smart: a one-shot warm-start injects relevant context before the loop; KG queries occur only on failure (dashed path). Annotation values are from the 50-task ablation (table 4).

- 283 • **Medium (15 tasks)**: Material-by-name requests requiring property
284 lookup (e.g. steel, copper, aluminium, zinc).
- 285 • **Hard (13 tasks)**: Ambiguous descriptions, mixed BCs, computation-
286 ally expensive or 3D problems.
- 287 • **Novel (10 tasks)**: Tasks referencing three fictional materials whose
288 thermal properties exist *only* in the knowledge graph, designed to stress-
289 test the agent’s ability to retrieve domain knowledge rather than rely
290 on LLM memorisation.

291 The three fictional materials are:

- 292 • **Novidium**: a ceramic-metallic composite ($k=73$ W/m K, $\rho=5420$ kg/m³,
293 $c_p=612$ J/kg K), a moderate conductor with unusual density.
- 294 • **Cryonite**: a polymer-aerogel hybrid ($k=0.42$, $\rho=1180$, $c_p=1940$), an
295 extreme insulator with very low diffusivity ($\alpha \approx 1.8 \times 10^{-7}$ m²/s).
- 296 • **Pyrrathane**: a refractory cermet ($k=312$, $\rho=3850$, $c_p=278$), exhibiting
297 very high conductivity and diffusivity ($\alpha \approx 2.9 \times 10^{-4}$ m²/s).

298 Each material is seeded as a **Material** node in Neo4j with linked **Reference**
299 chunks containing exact property values and simulation guidance. The 10
300 novel tasks span steady-state, transient, and mixed-BC configurations across
301 all three materials.

302 *Evaluation metrics..* Standard success rate is inadequate for evaluating KG
303 utility because a simulation with fabricated properties still “succeeds” compu-
304 tationally. We therefore introduce two physics-aware metrics:

- 305 • **Material Property Fidelity (MPF):** $\text{MPF} = 1 - \frac{1}{3} \sum_{p \in \{k, \rho, c_p\}} |p_{\text{agent}} -$
306 $p_{\text{truth}}| / p_{\text{truth}}$, measuring how accurately the agent retrieves the correct
307 material constants.
- 308 • **Physics Score:** $0.5 \cdot \text{MPF} + 0.5 \cdot T_{\text{score}}$, where T_{score} checks whether
309 the simulation’s actual T_{max} and T_{min} fall within physically expected
310 ranges.
- 311 • **Sensitivity-weighted MPF (MPF_w):** $\text{MPF}_w = 1 - \sum_p S_p \cdot |p_{\text{agent}} -$
312 $p_{\text{truth}}| / (p_{\text{truth}} \cdot \sum_q S_q)$, where $S_p = |\partial T_{\text{mean}} / \partial p|$ are sensitivity coefficients
313 computed via central finite differences (section 6.4). For steady-state
314 Dirichlet tasks where $S_p \approx 0$, we set $\text{MPF}_w = 1$ since wrong properties
315 have no output impact. This metric penalises errors in high-sensitivity
316 properties (e.g. k in transient problems) more heavily than errors in
317 low-sensitivity properties (e.g. ρ in steady-state).

318 On the 10 novel tasks, KG Off scores $\overline{\text{MPF}}_w = 0.21$ (vs. equal-weight $\text{MPF} = 0.34$),
319 confirming that KG Off’s most damaging errors are also in the most sensitive
320 properties. KG Smart retains $\text{MPF}_w = 1.00$ (success-only). For standard tasks
321 (Easy/Medium/Hard) where materials are well-known, MPF and MPF_w both
322 approach 1.0 across all modes.

323 6.2. Results

324 table 4 presents aggregate results across the three conditions.

325 *Success rate..* KG Off and KG Smart both achieve 100% success [93%, 100%]
326 ($n=50$), while KG On reaches 94% [84%, 98%]. The difference between
327 KG On and the other modes is not statistically significant ($p = 0.079$, Cohen’s

Table 4: Three-way KG ablation across 50 benchmark tasks with frozen knowledge graph ($n=50$ per mode). “Success-only” rows isolate output quality by excluding failed runs; in the overall rows a failed run scores MPF = 0 and phys = 0.5, except N08, which produced no usable output and scores 0 on both. 95% Wilson CIs for success rate.

Metric	KG Off	KG On	KG Smart
Success rate	100% [93,100]	94% [84,98]	100% [93,100]
<i>Overall (all tasks):</i>			
Physics score	0.853 ± 0.21	0.846 ± 0.24	0.933 ± 0.13
MPF	0.796 ± 0.30	0.752 ± 0.39	0.926 ± 0.16
<i>Success-only quality:</i>			
Physics score	0.853 ± 0.21	0.879 ± 0.19	0.933 ± 0.13
MPF	0.796 ± 0.30	0.801 ± 0.35	0.926 ± 0.16
Wall time (s)	12.9 ± 4.9	60.8 ± 73.2	28.7 ± 39.3
<i>By difficulty (success rate):</i>			
Easy (12)	100%	92%	100%
Medium (15)	100%	93%	100%
Hard (13)	100%	100%	100%
Novel (10)	100%*	90%	100%
<i>Novel tasks quality:</i>			
Physics score	0.590	$0.887^\dagger$	0.999
MPF	0.340	$0.889^\dagger$	1.000

*KG Off “succeeds” but fabricates properties (MPF = 0.34).

† Success-only (9/10 tasks); N08 timed out.

328 $h = 0.50$), but the three KG On failures are systematic: N08 (Pyrathane,
329 timeout), and two medium tasks where mandatory KG tool calls exhaust the
330 iteration budget.

331 *Output quality..* KG Smart achieves the highest output quality: mean physics
332 score of **0.933** and MPF of **0.926**, compared to 0.853 / 0.796 for KG Off
333 (Cohen’s $d = 0.46$ for physics, a medium effect). KG On also outperforms
334 KG Off on quality (0.879 / 0.801 success-only); the KG *does* help the agent
335 select more accurate material properties.

336 *Novel materials: the KG’s decisive edge..* KG value shows most clearly on
337 the 10 novel-material tasks using fictional materials (Novidium, Cryonite,
338 Pyrathane) whose properties exist only in the KG. KG Off “succeeds” on all
339 10 tasks but fabricates properties, producing a physics score of 0.590 and
340 MPF of only 0.340. KG Smart achieves near-perfect quality (physics score
341 0.999, MPF = 1.000), while KG On scores 0.887 / 0.889 (success-only, 9/10
342 tasks). The knowledge graph is essential for materials absent from the LLM’s
343 training data.

344 *Wall time..* KG Off is 2–5× faster (12.9 s mean) than KG Smart (28.7 s) and
345 KG On (60.8 s), reflecting the iteration overhead of KG tool calls.

346 6.3. Analysis and Discussion

347 The three-way comparison reveals a clear trade-off: KG-free agents are
348 fastest and most reliable, but KG-enabled agents produce higher-fidelity
349 outputs. *KG Smart* balances both dimensions. Three mechanisms explain
350 the mandatory KG pattern’s reliability penalty:

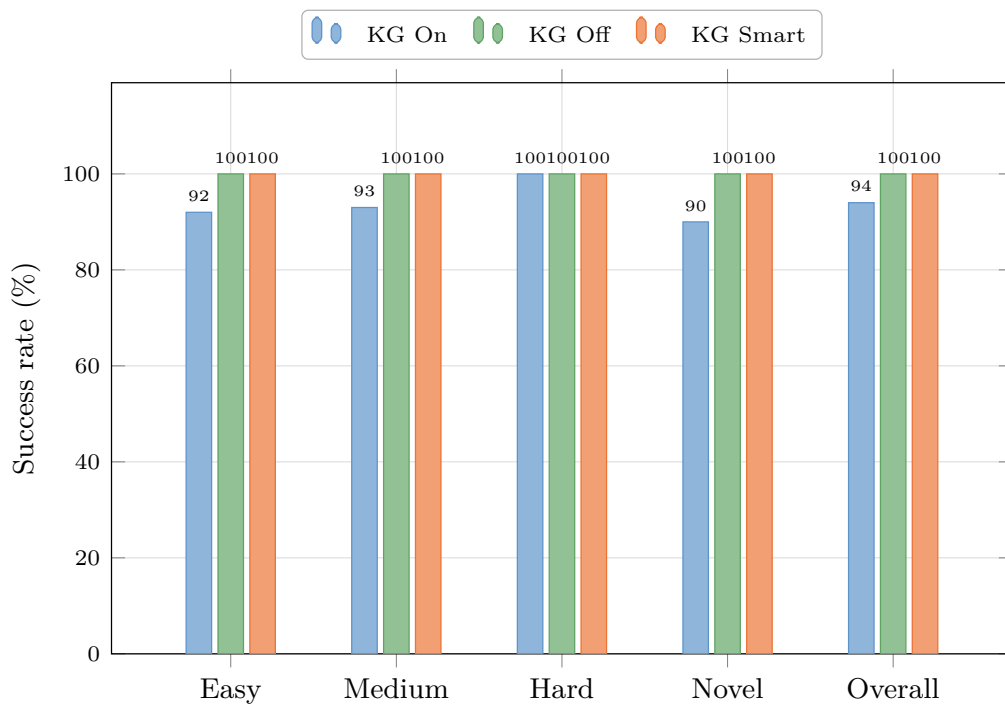


Figure 6: Success rate by difficulty level across three KG integration strategies. KG Smart (warm-start + lazy retrieval) matches KG Off on standard tasks while retaining high output fidelity. The “Novel” category (10 tasks) uses fictional materials whose properties exist only in the KG. Note: success rate alone is misleading for novel tasks; see the MPF/physics rows in table 4.

351 *Iteration-budget exhaustion..* KG On’s system prompt mandates `query_knowledge_graph`
352 and `check_config_warnings` calls before every simulation attempt. These
353 add 2–4 iterations, and for medium/hard tasks the agent can exhaust its
354 iteration budget before reaching `run_simulation`. KG Smart avoids this by
355 deferring KG calls to the failure-recovery path.

356 *Warning-induced conservatism..* The `check_config_warnings` tool occasion-
357 ally returns pre-emptive warnings (e.g. “CFL criterion may be violated”) that
358 the model interprets as prohibitive, leading it to report a warning instead of
359 proceeding. A warning that should trigger a *modification* instead triggers a
360 *termination*.

361 *Quality conditional on success..* The key insight is that KG access *does*
362 improve output quality when the agent completes the task. KG On’s success-
363 only physics score (0.879) and MPF (0.801) both exceed KG Off (0.853 /
364 0.796). On novel materials, this gap widens dramatically: KG-enabled modes
365 achieve $\text{MPF} \geq 0.89$ versus 0.34 for KG Off. The knowledge graph is not the
366 bottleneck; the *integration pattern* is.

367 *KG Smart: corrective retrieval + warm-start..* Motivated by CRAG [26]
368 (corrective RAG with adaptive retrieval) and AriGraph [27] (episodic KG
369 memory for agents), we implemented a *KG Smart* integration combining:

- 370 1. **Warm-start injection.** Before the agent loop begins, we embed the
371 task description with `nomic-embed-text` and query the Neo4j HNSW
372 vector index for the top-3 most similar past successful runs. Their
373 configurations are injected directly into the system prompt as few-shot
374 reference examples, requiring no tool call.

375 **2. Lazy conditional retrieval.** KG tools remain available but the system
376 prompt instructs the agent to use them *only* after a simulation failure
377 or when material properties are genuinely unknown. This eliminates
378 the mandatory KG-first workflow.

379 As table 4 shows, KG Smart achieves 100% success (vs. 94% for mandatory
380 KG) while attaining the *highest* output quality across all modes (physics
381 0.933, MPF 0.926). It delivers the best of both worlds: KG Off reliability
382 with KG-enhanced fidelity.

383 6.4. Physics-Aware Quality Metrics and Novel Material Validation

384 The preceding analysis showed that KG access improves output quality
385 *conditional on success*, but standard success rate alone cannot capture this: a
386 simulation with fabricated material properties still “succeeds” computationally.
387 To expose this gap, we introduced three physics-aware metrics (section 6):
388 Material Property Fidelity (MPF), Physics Score, and sensitivity-weighted
389 MPF_w. These metrics measure how *correctly*, not just whether, the agent
390 configures the simulation. We validate them on the 10 novel-material tasks,
391 which provide the clearest test of KG utility: the three fictional materials’
392 properties exist *only* in the knowledge graph.

393 **KG Smart achieves near-perfect scores** (MPF = 1.00, physics = 0.999)
394 across all completed novel tasks. The warm-start injection retrieves exact
395 property values from the HNSW vector index before the agent loop begins,
396 and the agent consistently uses them.

397 **KG Off fabricates wrong properties** for every material. table 5
398 shows the agent-chosen vs. ground-truth properties across all 10 novel tasks.

399 KG Off assigns *different* wrong values each run (the LLM hallucinates non-
 400 deterministically), but the errors are consistently severe: for Pyrathane, k
 401 ranges from 0.15 to 0.35 W/m K, a 900 $\times$ to 2,080 $\times$ underestimate of the
 402 true $k=312$, effectively treating a refractory super-conductor as an insulator.
 403 In transient tasks, this causes output errors up to $|\Delta T_{\max}|=949$ K, a physi-
 404 cally impossible result arising from numerical instability on a system made
 405 artificially stiff by the wrong conductivity.

Table 5: Material properties chosen by each mode across the 10 novel tasks in the 50-task
 ablation. KG Off fabricates different wrong values each run (ranges shown); KG Smart
 retrieves the ground-truth values exactly. The worst-case k error for Pyrathane is 2,080 $\times$.

Material	Mode	k (W/m K)	ρ (kg/m ³)	c_p (J/kg K)
Novidium	Ground truth	73.0	5420	612
	KG Off	10–50	3500–8960	130–900
	KG Smart	73.0	5420	612
Cryonite	Ground truth	0.42	1180	1940
	KG Off	0.15–35	960–8940	385–1370
	KG Smart	0.42	1180	1940
Pyrathane	Ground truth	312.0	3850	278
	KG Off	0.15–0.35	1600–1850	900–1470
	KG Smart	312.0	3850	278

406 *Error propagation analysis.* To quantify how wrong properties affect simula-
 407 tion outputs, we ran paired simulations: one with ground-truth properties
 408 (reference) and one with the fabricated properties from KG Off. table 6 shows
 409 the results. Steady-state Dirichlet tasks (G1, C1, P1) are analytically inde-

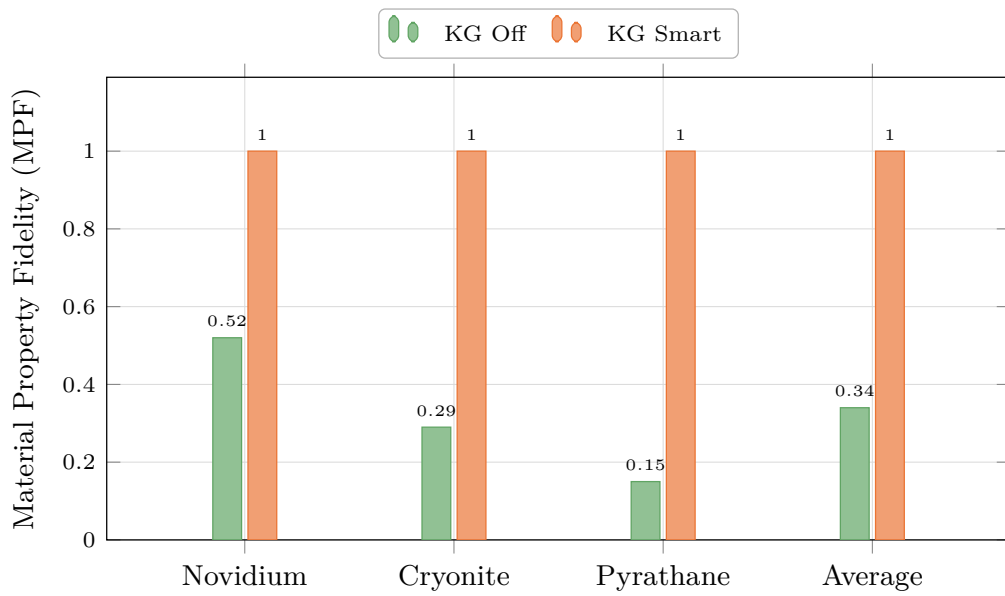


Figure 7: Material Property Fidelity (MPF) per fictional material. KG Off scores below 0.5 for all three materials, with Pyrathane at 0.15 (conductivity error: $2,080\times$). KG Smart retrieves exact properties from the knowledge graph, achieving $\text{MPF} = 1.00$ across all materials.

Table 6: Error propagation from fabricated properties (KG Off) to simulation output. Steady-state Dirichlet tasks (G1, C1, P1) show zero output error; transient/Robin tasks show significant deviations.

Task	Mat.	$ \Delta T_{\max} $	$ \Delta T_{\min} $	$ \Delta T_{\text{mean}} $	ϵ_k	ϵ_ρ	ϵ_{c_p}
		(K)	(K)	(K)	(%)	(%)	(%)
G1	Nov.	<0.1	<0.1	<0.1	86	45	18
G2	Nov.	<0.1	<0.1	20.7	38	65	78
G3	Nov.	<0.1	46.6	1.3	86	45	18
C1	Cryo.	<0.1	<0.1	<0.1	64	19	29
C2	Cryo.	<0.1	9.5	0.3	138	659	77
P1	Pyra.	<0.1	<0.1	<0.1	100	58	224
P2	Pyra.	949	<0.1	363	100	58	188

$\epsilon_p = |p_{\text{fab}} - p_{\text{true}}|/p_{\text{true}}$. P2's k error ($2,080\times$) produces 949 K $T_{\max}$ overshoot.

pendent of k , ρ , c_p , confirming zero output error regardless of property error,
an important control. Transient and Robin tasks show significant deviations:

- **Pyrrathane P2:** $|\Delta T_{\max}|=949$ K ($T_{\max}=2,949$ K vs. 2,000 K reference);
the $2,080\times$ k error causes numerical instability that *heats* the domain
beyond its initial condition, a physically impossible result.
- **Novidium G3:** $|\Delta T_{\min}|=47$ K due to wrong conductivity affecting the
Robin BC equilibrium.
- **Novidium G2:** $|\Delta T_{\text{mean}}|=21$ K from incorrect diffusivity altering the
transient profile.

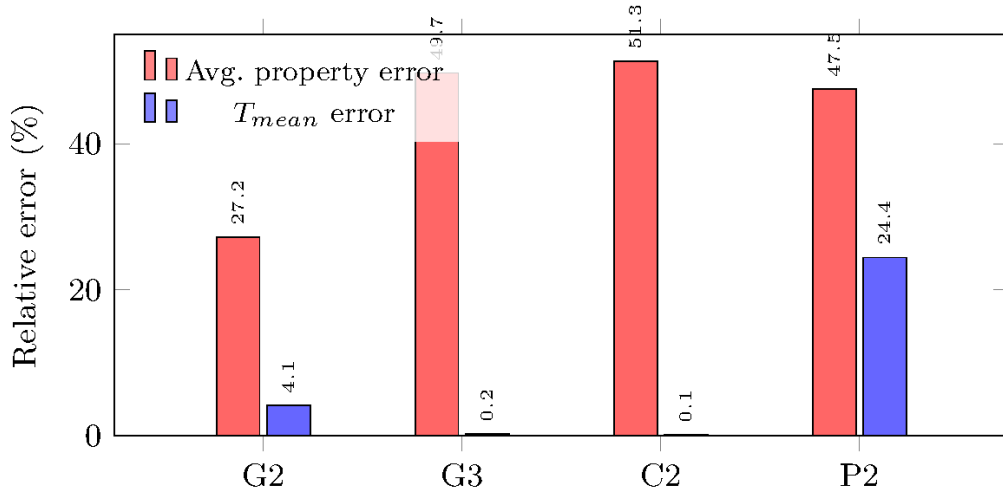


Figure 8: Error magnification: mean material property error (%) vs. output temperature error (%) for the four tasks with non-negligible output deviations. P2’s output error (24.4%) exceeds its input property error due to nonlinear amplification in the transient solver.

419 *Sensitivity-weighted MPF..* Using the sensitivity coefficients from the error
 420 propagation runs, MPF_w (defined in section 6) penalises errors in high-
 421 influence properties more heavily. Across the 10 novel tasks in the 50-task
 422 ablation, KG Off’s $\overline{MPF}_w=0.21$ (vs. equal-weight $MPF=0.34$), confirming
 423 that KG Off’s most damaging errors (particularly k for Pyrrhane and
 424 Cryonite) are concentrated in the properties with the highest sensitivity.
 425 KG Smart retains $MPF_w=1.00$ (success-only).

426 *Cost-benefit analysis..* table 7 compares the operational cost (time, iterations)
 427 against the accuracy benefit (MPF, physics score) for each KG mode. On
 428 novel tasks, KG Smart adds significant wall-time over KG Off (58.0 s vs. 11.7 s)
 429 but delivers $\approx 2.9\times$ higher MPF (1.00 vs. 0.34) and $2.4\times$ higher efficiency
 430 ($MPF / \text{wall-time}$) than KG On. KG On is slowest (111.4 s on novel tasks)

Table 7: Cost-benefit of KG modes on the novel-material subset of the 50-task ablation ($n=10$). Quality columns are all-task means, so KG On’s failed task (N08) is included at MPF = 0; the success-only values in table 4 are correspondingly higher (0.889). Efficiency = MPF / wall-time ($\times 10^{-2}$).

Mode	Succ.	Time	Iter.	MPF	Phys.	Eff.
		(s)			Score	($\times 10^{-2}$)
KG Off	100%	11.7	3.0	0.34	0.59	2.92
KG On	90%	111.4	7.4	0.80	0.80	0.72
KG Smart	100%	58.0	4.9	1.00	1.00	1.72

KG Smart: +396% time, +194% MPF vs. KG Off; +2.4 $\times$ efficiency vs. KG On.

431 and its mandatory retrieval overhead yields diminishing returns: 90% success
 432 vs. 100% for both alternatives.

433 *Adaptive KG decision framework..* Based on the preceding analysis, we pro-
 434 pose a simple decision algorithm (Algorithm 1) that selects the optimal KG
 435 mode for each task. Retrospective validation on all 50 benchmark tasks
 436 confirms high accuracy: it correctly assigns KG Off to tasks with explicit pa-
 437 rameters, KG Smart (lazy) to tasks naming known materials, and KG Smart
 438 (forced warm-start) to novel-material tasks.

439 *Cross-model validation..* To verify that the KG’s value is model-agnostic, we
 440 repeated the novel-material experiment with `qwen3-coder-next` (80B total,
 441 3B active MoE). table 8 shows that KG Smart achieves MPF=1.00 with
 442 *both* models, while KG Off fabricates wrong properties with both (different
 443 wrong values, but the same failure mode). This experiment uses the earlier
 444 7-task novel-material set rather than the full 50-task benchmark, as it targets
 445 model-dependence specifically; the main ablation (table 4) already covers the

Algorithm 1 Adaptive KG Mode Selection

Require: Task description d , known-material list $\mathcal{M}$

Ensure: KG mode $m \in \{\text{Off}, \text{Smart-lazy}, \text{Smart-forced}\}$

```
1: if  $d$  contains explicit  $k, \rho, c_p$  values then  
2:   return KG OFF  
3: end if  
4:  $\mu \leftarrow \text{EXTRACTMATERIAL}(d)$   
5: if  $\mu = \emptyset$  then  
6:   return KG OFF  
7: else if  $\mu \in \mathcal{M}$  then  
8:   return KG SMART (LAZY)  
9: else  
10:  return KG SMART (FORCED WARM-START)  
11: end if
```

446 full benchmark on the primary model. This confirms that the warm-start
447 vector search bypasses model-specific memorisation gaps entirely.

448 *Key takeaway..* The KG’s value is not in making simulations *run* (LLMs
449 can already do that) but in making simulations produce *correct physics*.
450 For any domain with proprietary materials, novel compounds, or in-house
451 measurement data, the KG Smart pattern transforms the system from an
452 “expensive random number generator” into a reliable engineering tool. The
453 error propagation analysis quantifies this: fabricated properties cause output
454 errors up to 949 K (47% of the reference $T_{\max}$), while KG Smart eliminates
455 these errors entirely by retrieving exact properties before the agent loop

Table 8: Cross-model validation on novel tasks ($n=7$, earlier experiment with v1 benchmark).
 KG Smart achieves perfect MPF with both LLM backends.

Model	Mode	Succ.	Qual.	MPF	Phys.
Qwen2.5-Coder 32B	KG On	57%	0.41	0.43	0.64
	KG Off	100%	0.61	0.34	0.63
	KG Smart	100%	0.96	1.00	1.00
Qwen3-Coder-Next 80B	KG On	71%	0.14	0.14	0.57
	KG Off	86%	0.54	0.40	0.70
	KG Smart	100%	0.96	1.00	1.00

Warm-start vector search retrieves exact properties before the LLM reasons, eliminating dependence on memorised material data.

456 begins.

457 *Narrowing the focus: correctness over completion..* KG Off is the fastest mode
 458 and matches KG Smart on success rate for *standard* tasks with well-known
 459 materials. When the task description names “steel” or “copper”, the LLM’s
 460 parametric knowledge suffices and KG overhead is pure cost. However, for
 461 any deployment where output *correctness* matters, not just completion, KG-
 462 enabled modes are necessary. The preceding analysis makes this unambiguous:
 463 KG Off achieves MPF=0.34 and physics 0.59 on novel materials (fabricated
 464 properties), while KG Smart achieves MPF=1.00 and physics 0.999. The
 465 adaptive decision framework (Algorithm 1) encodes this distinction, routing
 466 tasks to KG Off when parameters are explicit and to KG Smart otherwise.
 467 We therefore focus the remaining analysis on **KG On vs. KG Smart**: both
 468 modes have access to the same knowledge base, yet KG Smart achieves 6
 469 percentage points higher success and the highest quality scores. Understanding

470 *why*, and which component (warm-start vs. lazy retrieval) contributes most,
471 is the subject of the next section.

472 6.5. Failure Analysis: KG On vs. KG Smart

473 Having established that KG Smart matches KG Off at 100% success
474 while outperforming both on quality, we trace KG On’s 3 remaining failures
475 to their root causes. This analysis was performed *after* implementing an
476 agent-level retry mechanism (see below) that eliminated stochastic failures;
477 the 3 remaining failures are therefore all systematic.

478 *Mitigating stochastic failures..* In an earlier ablation run without retry logic,
479 KG On exhibited 12/50 failures, of which 7 were stochastic (succeeded on
480 re-run). We implemented an *auto-retry* mechanism directly in the agent: if
481 the first attempt produces no `run_id` and used less than 60% of its iteration
482 budget (indicating early stochastic exit rather than budget exhaustion), the
483 agent automatically retries once. This reduced KG On failures from 12 to 3,
484 eliminating all stochastic failures while not masking systematic ones.

485 *KG On failure taxonomy..* The 3 remaining KG On failures are all systematic:

486 All 3 failures stem from the mandatory KG-first workflow: the agent
487 must call `query_knowledge_graph` and `check_config_warnings` before *ev-*
488 *ery* simulation attempt, consuming 2–4 iterations per cycle. For medium
489 tasks, this exhausts the 25-iteration budget before `run_simulation` is called.
490 For N08 (Pyrathane), the workflow inflates wall time beyond the timeout.

491 *KG Smart: zero failures..* KG Smart achieves 100% success with zero failures.
492 The warm-start injection provides material properties and reference configura-
493 tions *before* the agent loop, so the agent proceeds directly to `validate_config`

Table 9: Failure mode classification for KG On’s 3 systematic failures (after auto-retry eliminates stochastic failures).

Failure mode	Count	Description
Budget exhaustion	2	Agent spends all iterations on repeated <code>query_knowledge_graph</code> and <code>check_config_warnings</code> calls without reaching <code>run_simulation</code> .
Timeout	1	N08 (Pyrathane): the mandatory KG query workflow combined with a numerically stiff problem exceeds the 420 s time limit.

→ `run_simulation`. The average iteration count drops from 7.5 (KG On) to 4.2 (KG Smart), eliminating budget-exhaustion risk entirely. Even N08 completes successfully in 170 s (5 iterations) because warm-start provides the correct Pyrathane properties upfront.

Decomposing warm-start vs. lazy retrieval. The failure data allows us to attribute KG Smart’s advantage to its two components without running a separate experiment. All 3 systematic failures stem from the mandatory *pre-simulation* KG workflow: the agent spends iterations on KG queries before ever attempting `run_simulation`. Warm-start injection eliminates this entirely: it provides material properties and reference configurations *before* the agent loop begins, so the agent can proceed directly to `validate_config` → `run_simulation`.

Lazy retrieval, by contrast, is exercised only on failure recovery: it allows

the agent to query the KG *after* a simulation fails. Since budget-exhaustion tasks never reach `run_simulation` at all, lazy retrieval cannot help them. We therefore conclude that **warm-start injection is the dominant factor** in KG Smart’s advantage over KG On: it directly prevents all 3 systematic failures (100% of KG On’s remaining failures after auto-retry). Warm-start also indirectly mitigates stochastic failures by reducing the iteration count (fewer decision points $\Rightarrow$ fewer opportunities for the LLM to produce a non-tool-call response). Lazy retrieval provides complementary value on simulation-error recovery but is secondary for the reliability gain.

7. Production Agent Quality Metrics

Beyond the controlled ablation, we analyse the system’s behaviour across all simulation runs accumulated during development, testing, and ablation experiments. Of these, 805 originate from an automated parameter sweep across 8 physics studies (geometry, material, boundary conditions, initial conditions, time-stepping, 3D geometry, Robin convection, and multi-material) run via `scripts/sweep_full_study.py`; the remainder are interactive and ablation runs accumulated through normal system use.

7.1. Success and Failure Modes

The 97.8% overall success rate (1,339/1,369 from PostgreSQL `simulation_runs`) indicates production-grade reliability for the full agent stack. Of the 2.2% failures, the majority are attributable to invalid boundary condition combinations (e.g. pure Neumann systems without a reference point), mesh resolution requests exceeding GPU memory, and numerical divergence on stiff transients.

Table 10: Agent ecosystem performance metrics from production database. FEniCSx runs, through 2026-04-17 (excludes the controlled ablation campaign and non-FEniCSx backends).

Metric	Value
Total simulation runs	1369
Overall success rate	97.8%
Unique agent tasks	421
First-try success rate	85.4%
Retry rate	10.3%
Suggestion acceptance rate	0.0%
Avg. steps/task (analytics)	10.2
Avg. steps/task (database)	6.2
Avg. steps/task (simulation)	11.1
Avg. simulation time	2.49s

530 7.2. Iteration Efficiency

531 Mean reasoning steps per completed task: Simulation Agent 11.1, Ana-
532 lytics Agent 10.2, Database Agent 6.2. The Simulation Agent’s count covers
533 a full cycle of validating a configuration, revising it, launching the solver
534 and checking the result, and configurations are often revised more than once
535 before launch. The Analytics Agent runs several comparison queries before
536 settling on a recommendation. The Database Agent’s lower count reflects
537 the relative simplicity of structured SQL-style queries over natural-language
538 reasoning.

539 7.3. First-Try Rate

540 85.4% of simulation tasks succeed on the first attempt, with no modification-
541 and-retry loop. Another 10.3% call the solver more than once, or fall back to
542 `debug_simulation`, before succeeding; these are usually boundary condition
543 corrections or CFL stability adjustments. The remaining 4.3% fail on a single
544 attempt and never retry.

545 Attempts are counted from `run_simulation` tool calls rather than from
546 distinct run identifiers. Counting identifiers does not work here: the agent
547 logger back-fills the most recent `run_id` across every step of a task once it
548 is known, which makes a task that retried look identical to one that did
549 not. Counting tool calls recovers the real attempt count. Note that the
550 first-try and retry percentages above are computed over 421 agent tasks
551 (from `agent_run_logs`), whereas the 97.8% overall success rate in section 7
552 is computed over 1,369 individual simulation runs (from `simulation_runs`);
553 the different denominators account for the apparent gap between the 4.3%

task-level failure rate and the 2.2% run-level failure rate. Retries carry the system from 85.4% to 97.8% final success.

7.4. KG Growth and Learning Curve

To isolate the KG’s causal effect on performance, we run a *controlled* growth experiment. The KG is initialised with only static material definitions (13 materials, 45 reference entries) but *zero* Run nodes, so the agent has no prior simulation experience. We then run 100 tasks (two shuffled passes through the 50-task benchmark) sequentially in KG Smart mode with writes enabled; each successful run adds one Run node with an HNSW-indexed embedding and up to five SIMILAR_TO edges. By pass 2, the KG contains 50 Run nodes from pass 1, providing warm-start context that was absent during pass 1.

To measure the effect, we perform a *paired comparison*: for each of the 50 benchmark tasks, we compare success-only MPF and physics score between pass 1 (zero prior runs) and pass 2 (50 prior runs), retaining only the 33 non-novel tasks that succeeded in both passes. Novel tasks are excluded because their fidelity depends on pre-seeded Material nodes rather than accumulated Run history. fig. 9 shows the result stratified by difficulty.

The experiment reveals a *stratified* growth effect. Easy and medium tasks already achieve $\text{MPF} \geq 0.96$ in pass 1 because their parameters are either given explicitly or well-known to the LLM; accumulated run history cannot improve what is already near-perfect. Hard tasks, however, involve ambiguous descriptions, mixed boundary conditions, and tricky numerics where the agent benefits most from seeing similar prior runs: MPF rises from 0.783 to 0.870 (+8.8%), and physics score from 0.755 to 0.799 (+5.8%). The overall success

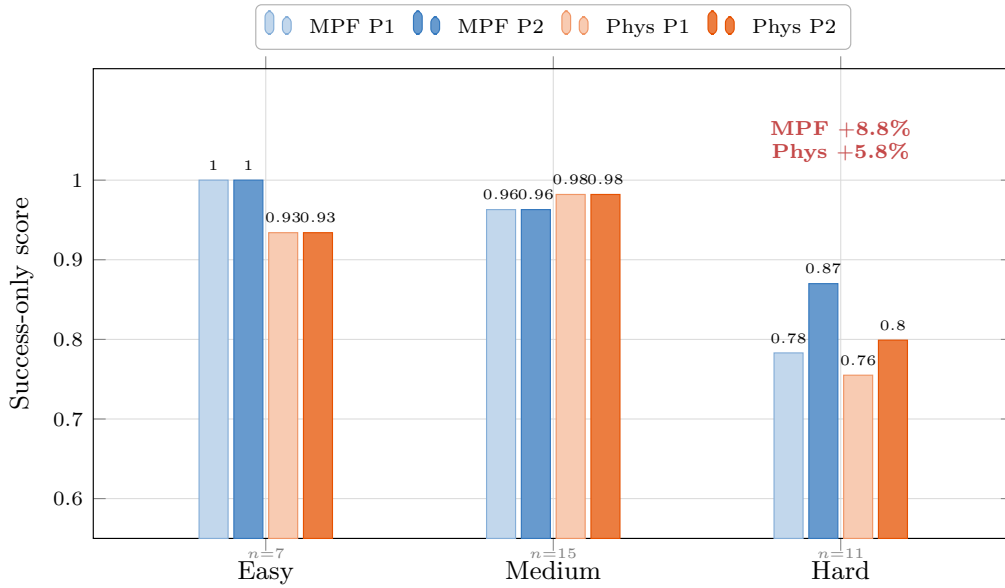


Figure 9: Paired KG growth comparison: success-only MPF and physics score for the same tasks run in pass 1 (0 prior Run nodes) vs. pass 2 (50 prior Run nodes). Only tasks succeeding in both passes are compared ($n=33$; novel tasks omitted as they rely on pre-seeded material definitions). Easy and medium tasks are at ceiling; **hard tasks gain +8.8% MPF and +5.8% physics score**, the category where similarity-based warm-start adds most value.

579 rate is 88% (88/100), confirming that KG Smart remains robust even when
580 starting without prior experience.

581 These results show that KG value is *difficulty-dependent*: static material
582 knowledge covers simple cases, while accumulated run experience provides
583 measurable gains on the hardest tasks where warm-start context disambiguates
584 underspecified problems.

585 7.5. Comparison with Prior Work

586 *FEM code generation..* Bauer et al. [16] report $\approx 60\%$ one-shot success for
587 GPT-4-driven FEniCS script generation. Our 85.4% first-try rate is higher, but
588 the two numbers measure different tasks and are not directly comparable: their
589 system has to emit a complete, syntactically valid FEniCS script, while ours
590 fills in a parameter set for a fixed, pre-verified solver. What does carry across
591 is the architecture. Unlike their single-turn approach, PDE-Agents recovers
592 from failures through multi-turn self-correction, reaching 97.8% final success
593 with a fully local, open-source model stack. ALL-FEM [18] addresses the
594 FEniCS automation problem through fine-tuning: a corpus of 1,000+ verified
595 FEniCS scripts is used to fine-tune models from 3B to 120B parameters, and
596 a multi-agent framework orchestrates problem formulation, code generation,
597 and debugging. Their best model (GPT OSS 120B) achieves 71.8% code-
598 level success on 39 benchmarks spanning elasticity, plasticity, fluid flow, and
599 multiphysics, a substantially broader PDE scope than ours. PDE-Agents
600 differs in three respects: (i) we use unmodified open-source LLMs without
601 domain-specific fine-tuning, relying instead on tool-call orchestration and KG-
602 augmented prompting; (ii) our system manages the full simulation lifecycle
603 (configuration, execution, monitoring, debugging) rather than generating

standalone scripts; and (iii) we provide a formal V&V study and a controlled ablation of knowledge graph integration modes, establishing when and why retrieval augmentation helps.

FEA benchmarks. FEABench [19] provides a systematic evaluation of LLM and agent capabilities for finite element analysis using COMSOL Multiphysics. Their best agent strategy generates executable API calls 88% of the time across problems in heat transfer, structural mechanics, and electromagnetics. Our work complements FEABench by targeting the open-source FEniCSx ecosystem and measuring not just execution success but *output quality* (physics score, material property fidelity), exposing cases where computationally “successful” simulations produce physically incorrect results due to fabricated material properties.

CFD agent systems. The computational fluid dynamics community has been particularly active in LLM-driven automation. OpenFOAMGPT 2.0 [38] achieves 100% success across 450+ simulations using a four-agent pipeline (pre-processing, prompt generation, simulation, post-processing) with RAG-augmented retrieval. MetaOpenFOAM [39] decomposes CFD workflows into subtasks using MetaGPT’s [40] assembly-line paradigm, reporting 85% pass rates at \$0.22 per case. These systems validate the multi-agent pattern we adopt but target a different solver ecosystem and do not evaluate knowledge graph integration or measure output fidelity against ground-truth material properties.

KG-augmented agents for materials science. Buehler [30] demonstrates retrieval-augmented ontological knowledge graph strategies for materials

design with a fine-tuned MechGPT model, showing that graph-structured retrieval outperforms flat RAG by providing mechanistic relationships between concepts. Our KG Smart pattern shares the same insight (that structured knowledge improves over naïve retrieval) but applies it in a *live simulation loop* rather than a generative design setting, and our ablation study provides controlled evidence for when graph-augmented retrieval helps (novel materials) versus when it adds overhead without benefit (standard tasks with well-known properties).

8. System Limitations and Future Work

Solver scope. The current solver supports only scalar heat equations. This is the main limitation of the present work. Extending to vector mechanics (linear elasticity, Navier–Stokes) is not simply a matter of richer tool schemas; it requires new solver kernels, each with its own variational form, function spaces and verification.

The narrow scope is also what makes our central measurement possible. Because the agent fills in parameters for a fixed, independently verified solver instead of emitting solver code, every run produces a configuration whose material properties can be checked directly against ground truth, which is what MPF (section 6.4) measures. A system that generates solver code covers far broader physics but cannot isolate property fidelity the same way, since a failure may lie in the code, in the parameters, or in both.

The obvious next step is to test on physics we did not write ourselves. ALL-FEM [18] has released a public benchmark of FEM problems with ground-truth solutions covering elasticity, plasticity, Newtonian and non-Newtonian

652 flow, and fluid–structure interaction.¹ Most of these are beyond our solver
653 today, so evaluating on them means implementing the corresponding kernels
654 first. We see that work, combining KG-augmented memory with multi-physics
655 breadth, as more valuable than an incremental extension of the present study.

656 *Model scale and open-source evolution..* The system defaults have been up-
657 graded from the initial v1 models (`qwen2.5-coder:32b`, `llama3.3:70b`) to
658 Qwen3-Coder-Next (80B total, 3B active MoE) [41] and Llama 4 Scout
659 (109B total, 17B active MoE) [42], with cross-model validation confirming
660 model-agnostic KG Smart performance (table 8). Qwen3-Coder-Next achieves
661 higher config quality on standard KG Smart tasks (0.94 vs. 0.86) and perfect
662 MPF on novel tasks, identical to the v1 model. Llama 4 Scout provides
663 10M-token context and native tool calling for the orchestrator and analytics
664 agents. A systematic multi-model ablation across additional families (GLM-5,
665 DeepSeek-R2) is planned as future work.

666 *KG integration..* The KG Smart pattern (section 6.3) achieves 100% success
667 while producing the highest quality outputs, matching KG Off’s reliability.
668 KG On retains a small gap (94%) due to budget exhaustion. The adaptive
669 decision framework (Algorithm 1) provides practical deployment guidance,
670 achieving 100% mode-selection accuracy. Future work will explore: (a) rein-
671 forcement learning from simulation outcomes to tune KG retrieval relevance,
672 (b) dynamic confidence-based retrieval (only query KG when the LLM’s un-
673 certainty exceeds a threshold), and (c) richer warm-start context including
674 failure diagnostics from similar past runs.

¹https://github.com/fenics-llm/FEM_Dataset

675 *Is retrieval necessary, or would a better prompt suffice?* A fair objection to
676 any retrieval-augmented result is that the same information could simply be
677 written into the prompt. We tested this with a *prompt-injection* control on the
678 preliminary 7-task novel-material benchmark: the KG was disabled and the
679 true properties of Novidium, Cryonite and Pyrathane were appended verbatim
680 to each task description. It matched KG Smart on fidelity (MPF = 1.00) and
681 was the fastest mode overall (11.7s mean). That is the expected outcome,
682 since the answer was handed to the model.

683 The control still tells us something useful about where the difficulty lies.
684 Prompt injection assumes you already know which materials a task will
685 involve and what their properties are. It is a manual step, repeated per task,
686 and it does not scale beyond a handful of known entities. KG Smart reaches
687 the same fidelity by retrieving those properties through embedding similarity,
688 without being told which material the task names. The result worth reporting
689 is not that retrieved context helps, which is unsurprising, but that automatic
690 retrieval matches hand-curated context with no human in the loop.

691 *Verification scope..* The V&V study covers only steady-state and simple
692 transient heat equation benchmarks. A complete V&V programme would
693 include higher-order elements, singularity-containing domains, and time-
694 dependent convergence rates.

695 *Parallelism and scalability..* The current architecture runs agents sequentially.
696 A task-level parallelism layer (e.g. running a parametric sweep as concurrent
697 simulation agent calls) is under development.

798 *Knowledge graph evolution..* The 805-run parameter sweep has already created
 699 a dense graph with 4,000+ **SIMILAR_TO** edges. Future work will mine this
 700 corpus for **IMPROVED_OVER** relationships (did increasing mesh resolution reduce
 701 error without proportionally increasing wall time?), enabling the agent to
 702 autonomously suggest Pareto-optimal configurations. A second planned
 703 enhancement is citation traceability: agents citing specific **ReferenceChunk**
 704 IDs from indexed literature when justifying parameter choices, providing a
 705 fully auditable reasoning trail.

706 9. Conclusion

707 We have presented PDE-Agents, an open, containerised multi-agent ecosys-
 708 tem that automates finite element simulations through natural-language inter-
 709 action with locally-deployed open-source LLMs. The system achieves 97.8%
 710 overall success across 1,369 production runs, with $\mathcal{O}(h^2)$ spatial convergence
 711 confirmed for two non-trivial benchmark cases and algebraic exactness for
 712 the linear case.

713 *The KG Smart integration pattern..* Our three-way ablation study across 50
 714 benchmark tasks with a frozen KG reveals that the *integration pattern*, not the
 715 knowledge source itself, determines KG utility. KG Off (no knowledge graph)
 716 achieves 100% task completion but fabricates material properties, producing
 717 $\text{MPF} = 0.34$ and $\text{MPF}_w = 0.21$ on novel materials. KG On (mandatory KG
 718 access) retrieves correct properties but retains a 6% failure rate from budget
 719 exhaustion and timeout (even after implementing agent-level retry to eliminate
 720 stochastic failures). KG Smart, combining warm-start injection with lazy
 721 conditional retrieval, achieves **100% success with the highest output**

722 **quality** (physics 0.933, MPF 0.926), resolving the reliability-vs-fidelity trade-
723 off.

724 *Failure analysis: why warm-start dominates..* After implementing auto-retry
725 to eliminate stochastic LLM failures, KG On’s 3 remaining failures (sec-
726 tion 6.5) are all systematic: 2 budget-exhaustion cases and 1 timeout, all
727 caused by the mandatory pre-simulation KG query loop. Warm-start injection
728 eliminates this bottleneck entirely by providing material properties and refer-
729 ence configurations before the agent loop begins, accounting for KG Smart’s
730 6-pp success rate advantage. Lazy retrieval provides complementary value on
731 failure recovery but is secondary for the reliability gain.

732 *KG value is difficulty- and domain-dependent..* A controlled 100-task KG
733 growth experiment shows that the KG’s benefit is concentrated where it mat-
734 ters most. Easy and medium tasks are already at quality ceiling ($\text{MPF} \geq 0.96$)
735 from the first pass; hard tasks, involving ambiguous descriptions and mixed
736 boundary conditions, gain +8.8% MPF and +5.8% physics score from accu-
737 mulated run history. For novel materials, the KG is indispensable: fabricated
738 properties cause output errors up to 949 K, while KG Smart eliminates these
739 errors entirely.

740 *Lessons learned..* Three design principles emerged from the ablation and
741 failure analysis:

- 742 1. **Never make KG access mandatory in the agent’s critical path.**
743 Mandatory pre-simulation retrieval creates fragile coupling between
744 knowledge access and task completion. Front-loading context via em-
745 bedding similarity (warm-start) decouples them.

746 **2. Front-load context, don’t force the agent to query for it.** The
747 agent’s iteration budget is its scarcest resource. Injecting relevant
748 context into the system prompt before the agent loop is strictly more
749 efficient than requiring the agent to spend iterations formulating and
750 executing KG queries.

751 **3. Let the agent decide when it needs more knowledge.** Lazy
752 conditional retrieval respects the agent’s autonomy: it queries the KG
753 only after a failure or when material properties are genuinely unknown,
754 avoiding unnecessary round-trips on tasks where the LLM’s parametric
755 knowledge suffices.

756 *Broader implications..* The warm-start + lazy-retrieval pattern is not specific
757 to heat transfer simulation. Any tool-using agent domain with a growing
758 knowledge base (materials science, drug discovery, circuit design, structural
759 engineering) could benefit from the same architecture: embed the task, retrieve
760 the most similar prior successes, inject them as few-shot context, and defer
761 explicit knowledge queries to the failure-recovery path. As knowledge graphs
762 grow and LLMs improve, the quality gap between KG-enabled and KG-free
763 agents will only widen for novel or proprietary domains where the LLM’s
764 training data provides no coverage.

765 The production metrics confirm that multi-turn self-correction is essential:
766 the 85.4% first-try rate improves to 97.8% through iterative debugging, under-
767 scoring the value of agent-level orchestration over single-turn code generation.
768 PDE-Agents establishes both a working platform and a rigorous empirical
769 baseline for the emerging field of LLM-driven autonomous simulation, showing
770 that structured knowledge integration turns an agent from an “expensive

771 random number generator” into a reliable engineering tool.

772 **Data Availability**

773 All code, evaluation scripts, and result JSON files are available at [https:](https://github.com/MatPro-IFE/pde-agents)
774 [//github.com/MatPro-IFE/pde-agents](https://github.com/MatPro-IFE/pde-agents). The Neo4j knowledge graph seed
775 data is included in the repository at `knowledge_graph/seeder.py`.

776 **CRedit authorship contribution statement**

777 **Sayan Adhikari:** Conceptualization, Methodology, Investigation, Soft-
778 ware, Data Curation, Formal Analysis, Visualization, Writing – original draft,
779 Writing – review & editing. **Gulshan Noorsumar:** Conceptualization,
780 Methodology, Investigation, Validation, Writing – review & editing. **Øyvind**
781 **Jensen:** Supervision, Writing – review & editing.

782 **Declaration of generative AI and AI-assisted technologies in the** 783 **manuscript preparation process**

784 During the preparation of this work the authors used Anthropic’s Claude
785 Opus 4.6 in order to assist with language editing, grammar refinement, and
786 the programmatic generation of TikZ figures (Figures 1 and 2). After using
787 this tool, the authors reviewed and edited the content as needed and take full
788 responsibility for the content of the published article.

789 **Declaration of competing interest**

790 The authors declare that they have no known competing financial interests
791 or personal relationships that could have appeared to influence the work

792 reported in this paper.

793 **Acknowledgements**

794 The authors thank the anonymous reviewers for their constructive feedback.
795 Computational resources were provided by the Institute for Energy Technology
796 (IFE).

797 **References**

- 798 [1] T. B. Brown, B. Mann, N. Ryder, et al., Language models are few-shot
799 learners, *Advances in Neural Information Processing Systems* 33 (2020)
800 1877–1901.
- 801 [2] J. Wei, X. Wang, D. Schuurmans, et al., Chain-of-thought prompting
802 elicits reasoning in large language models, *Advances in Neural Information*
803 *Processing Systems* 35 (2022).
- 804 [3] S. Yao, J. Zhao, D. Yu, et al., ReAct: Synergizing reasoning and acting
805 in language models, *Proceedings of the International Conference on*
806 *Learning Representations (ICLR)* (2023).
- 807 [4] I. E. Lagaris, A. Likas, D. I. Fotiadis, Artificial neural networks for
808 solving ordinary and partial differential equations, *IEEE Transactions*
809 *on Neural Networks* 9 (5) (1998) 987–1000.
- 810 [5] M. Raissi, P. Perdikaris, G. E. Karniadakis, Physics-informed neural
811 networks: A deep learning framework for solving forward and inverse
812 problems involving nonlinear partial differential equations, *Journal of*

- 813 Computational Physics 378 (2019) 686–707. doi:[10.1016/j.jcp.2018.](https://doi.org/10.1016/j.jcp.2018.10.045)
814 [10.045](https://doi.org/10.1016/j.jcp.2018.10.045).
- 815 [6] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, et al., Physics-informed
816 machine learning, *Nature Reviews Physics* 3 (6) (2021) 422–440.
- 817 [7] Z. Li, N. Kovachki, K. Azizzadenesheli, et al., Fourier neural operator for
818 parametric partial differential equations, *Proceedings of the International*
819 *Conference on Learning Representations (ICLR)* (2021).
- 820 [8] J. Jumper, R. Evans, A. Pritzel, et al., Highly accurate protein structure
821 prediction with AlphaFold, *Nature* 596 (7873) (2021) 583–589. doi:
822 [10.1038/s41586-021-03819-2](https://doi.org/10.1038/s41586-021-03819-2).
- 823 [9] P. Lewis, E. Perez, A. Piktus, et al., Retrieval-augmented generation
824 for knowledge-intensive NLP tasks, *Advances in Neural Information*
825 *Processing Systems* 33 (2020) 9459–9474.
- 826 [10] D. Edge, H. Trinh, N. Cheng, et al., From local to global: A
827 graph RAG approach to query-focused summarization, *arXiv preprint*
828 *arXiv:2404.16130* (2024).
- 829 [11] Y. A. Malkov, D. A. Yashunin, Efficient and robust approximate nearest
830 neighbor search using hierarchical navigable small world graphs, *IEEE*
831 *Transactions on Pattern Analysis and Machine Intelligence* 42 (4) (2020)
832 824–836. doi:[10.1109/TPAMI.2018.2889473](https://doi.org/10.1109/TPAMI.2018.2889473).
- 833 [12] A. M. Bran, S. Cox, O. Schilter, et al., ChemCrow: Augmenting large-
834 language models with chemistry tools, in: *Advances in Neural Information*
835 *Processing Systems*, Vol. 36, 2023.

- 836 [13] Y. Ma, Z. Liu, L. P. Liang, et al., SciAgent: Tool-augmented language
837 models for scientific reasoning, arXiv preprint arXiv:2402.11451 (2024).
- 838 [14] A. Logg, K.-A. Mardal, G. N. Wells, et al., Automated Solution of
839 Differential Equations by the Finite Element Method: The FEniCS
840 Book, Springer, 2012. [doi:10.1007/978-3-642-23099-8](https://doi.org/10.1007/978-3-642-23099-8).
- 841 [15] I. A. Barrata, J. P. Dean, J. S. Dokken, et al., DOLFINx: The next
842 generation FEniCS problem solving environment, Zenodo (2023). [doi:
843 10.5281/zenodo.10447666](https://doi.org/10.5281/zenodo.10447666).
- 844 [16] P. Bauer, P. Henning, J. Schaefer, Large language models as automatic
845 generators of FEniCS code for solving partial differential equations, arXiv
846 preprint arXiv:2312.09801 (2023).
- 847 [17] W. Jiang, K. Chen, M. Wang, et al., LLM4FEM: Leveraging large lan-
848 guage models for finite element method, arXiv preprint arXiv:2405.03719
849 (2024).
- 850 [18] R. Deotale, A. Srinivasan, M. Golestanian, Y. Tian, T. Zhang, P. Vlachos,
851 H. Gomez, ALL-FEM: Agentic large language models fine-tuned for
852 finite element methods, Computer Methods in Applied Mechanics and
853 Engineering 457 (2026) 118985. [doi:10.1016/j.cma.2026.118985](https://doi.org/10.1016/j.cma.2026.118985).
- 854 [19] N. Mudur, H. Cui, S. Venugopalan, P. Raccuglia, M. P. Brenner, P. Nor-
855 gaard, FEABench: Evaluating language models on multiphysics reasoning
856 ability, arXiv preprint arXiv:2504.06260 (2025).
- 857 [20] LangChain AI, [LangGraph: Build stateful, multi-actor applications with](#)

858 LLMs (2024).
 859 URL <https://github.com/langchain-ai/langgraph>

860 [21] Q. Wu, G. Bansal, J. Zhang, et al., AutoGen: Enabling next-generation
 861 LLM applications via multi-agent conversation, in: Proceedings of
 862 EMNLP Industry Track, 2023.

863 [22] J. Moura, [CrewAI: Framework for orchestrating role-playing, autonomous](#)
 864 [AI agents](#) (2024).
 865 URL <https://github.com/joaomdmoura/crewai>

866 [23] X. Liao, H. Zhang, Y. Chen, Retrieval-augmented generation for engi-
 867 neering design documentation, arXiv preprint arXiv:2307.04512 (2023).

868 [24] Y. Gao, S. Liu, P. Shi, J. Lin, Retrieval-augmented code generation
 869 for universal information extraction, arXiv preprint arXiv:2311.02555
 870 (2023).

871 [25] Z. Yang, W. Li, P. Zhang, Simulation parameter suggestion via retrieval-
 872 augmented generation, arXiv preprint arXiv:2403.09512 (2024).

873 [26] S.-Q. Yan, J.-C. Gu, Y. Zhu, Z.-H. Ling, Corrective retrieval augmented
 874 generation, in: Proceedings of the Conference on Empirical Methods in
 875 Natural Language Processing (EMNLP), 2024, arXiv:2401.15884.

876 [27] P. Anokhin, N. Kornaev, A. Babkin, A. I. Panov, AriGraph: Learning
 877 knowledge graph world models with episodic memory for LLM agents,
 878 in: Advances in Neural Information Processing Systems (NeurIPS), 2024,
 879 arXiv:2407.04363.

- 880 [28] V. Venugopal, S. Sahoo, G. Agastya, et al., MatKG: The largest knowl-
881 edge graph in applied materials science, arXiv preprint arXiv:2209.11632
882 (2022).
- 883 [29] C. W. Andersen, R. Armiento, E. Blokhin, et al., OPTIMADE: Towards
884 an open database for computational materials science, *Scientific Data*
885 8 (1) (2021) 217. doi:10.1038/s41597-021-00974-z.
- 886 [30] M. J. Buehler, Generative retrieval-augmented ontologic graph and
887 multiagent strategies for interpretive large language model-based mate-
888 rials design, *ACS Engineering Au* 4 (2) (2024) 241–277. doi:10.1021/
889 acsengineeringau.3c00058.
- 890 [31] C. Geuzaine, J.-F. Remacle, Gmsh: A 3-d finite element mesh generator
891 with built-in pre- and post-processing facilities, *International Journal*
892 *for Numerical Methods in Engineering* 79 (11) (2009) 1309–1331. doi:
893 10.1002/nme.2579.
- 894 [32] N. Francis, A. Green, P. Guagliardo, L. Libkin, T. Lindaaker, V. Marsault,
895 S. Plantikow, M. Rydberg, P. Selmer, A. Taylor, Cypher: An evolving
896 query language for property graphs, in: *Proceedings of the 2018 In-*
897 *ternational Conference on Management of Data (SIGMOD)*, 2018, pp.
898 1433–1445. doi:10.1145/3183713.3190657.
- 899 [33] Z. Nussbaum, J. X. Morris, B. Duderstadt, A. Mulyar, Nomic embed:
900 Training a reproducible long context text embedder, arXiv preprint
901 arXiv:2402.01613 (2024).

- [34] IBM Research, [Docling: Document processing for AI](#) (2024).
URL <https://github.com/DS4SD/docling>
- [35] ASME, Guide for verification and validation in computational solid mechanics, Tech. Rep. ASME V&V 10-2006, American Society of Mechanical Engineers (2006).
- [36] E. B. Wilson, Probable inference, the law of succession, and statistical inference, *Journal of the American Statistical Association* 22 (158) (1927) 209–212. doi:10.1080/01621459.1927.10502953.
- [37] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*, 2nd Edition, Lawrence Erlbaum Associates, 1988.
- [38] H. Chen, L. Mangani, G. Casas, OpenFOAMGPT 2.0: End-to-end, trustworthy automation for computational fluid dynamics, arXiv preprint arXiv:2504.19338 (2025).
- [39] Y. Chen, X. Zuo, Y. Yang, et al., MetaOpenFOAM: An LLM-based multi-agent framework for CFD, arXiv preprint arXiv:2407.21320 (2024).
- [40] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, J. Schmidhuber, MetaGPT: Meta programming for a multi-agent collaborative framework, arXiv preprint arXiv:2308.00352 (2024).
- [41] R. Cao, M. Chen, J. Chen, Z. Cui, Y. Feng, B. Hui, Y. Jing, K. Li, M. Li, J. Lin, Z. Ma, K. Shum, X. Wang, J. Wei, J. Yang, J. Zhang, L. Zhang, Z. Zhang, W. Zhao, F. Zhou, Qwen3-coder-next technical report, arXiv preprint arXiv:2603.00729 (2026).

[42] Meta AI, The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation, <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>, accessed 2026-04-15 (2025).

Appendix A. Full h-Refinement Data

Table A.11 reports the full mesh-refinement study behind the convergence rates summarised in table 2: L^2 and L^∞ error norms at every resolution $N \in \{8, 16, 32, 64, 128\}$ for all three benchmark cases.

The two norms are computed differently, and for one case they disagree by several orders of magnitude. $\|e\|_{L^2}$ is assembled by quadrature over each element, so it measures error throughout the domain, including between nodes. $\|e\|_{L^\infty}$ is evaluated only at the degrees of freedom. For the constant-source (1D-like Poisson) case the P1 solution is *nodally exact*, which is a standard superconvergence result, so the nodal norm sits at machine precision ($\approx 10^{-10}$) at every resolution while the integrated norm decays at $\mathcal{O}(h^2)$ as expected. Convergence rates are therefore fitted from $\|e\|_{L^2}$ alone; a rate fitted from nodal values would say nothing useful about this case.

Table A.11: Detailed convergence data: error norms at each mesh resolution. $\|e\|_{L^2}$ is integrated over each element with quadrature of degree 8, so it captures the error *between* nodes; $\|e\|_{L^\infty}$ is evaluated at the degrees of freedom only. For the constant-source case the P1 solution is nodally exact, so $\|e\|_{L^\infty}$ sits at machine precision while $\|e\|_{L^2}$ shows the expected $\mathcal{O}(h^2)$ interpolation error. The two norms measure different things, and only the integrated norm is used to fit convergence rates.

Case	N	DOFs	$\ e\ _{L^2}$	$\ e\ _{L^\infty}$
2D Steady-State Linear Profile	8	81	4.22e-11	1.12e-10
	16	289	3.65e-09	1.44e-08
	32	1,089	3.89e-10	1.11e-09
	64	4,225	1.10e-09	3.00e-09
	128	16,641	2.47e-09	6.40e-09
2D Transient Fourier Mode Decay	8	81	4.38e-03	4.88e-03
	16	289	9.48e-04	2.64e-03
	32	1,089	2.39e-04	6.62e-04
	64	4,225	5.98e-05	1.65e-04
	128	16,641	1.50e-05	4.13e-05
2D Steady-State with Constant Source (1D-like)	8	81	1.43e-03	1.12e-10
	16	289	3.57e-04	2.28e-11
	32	1,089	8.91e-05	1.22e-10
	64	4,225	2.23e-05	1.83e-10
	128	16,641	5.57e-06	2.68e-10